

Lecture 3: Conditionals, Loops, and Pattern Printing

Foundation

1. Input/Output & Conditionals

Understanding how a program makes decisions and interacts with the user is the first step toward writing dynamic logic.

- **Standard Input (cin):** The `cin` command extracts user input from the standard input stream (keyboard) and stores it in a declared variable.
- **Whitespace Ignorance:** By default, `cin` completely ignores whitespace characters (Space, Tab, Enter/`\n`). If you specifically need to read a space or enter key stroke as a character, you must use `cin.get()`.
- **Execution Branching (if-else):** Conditionals allow a program to branch its execution path. The compiler evaluates the condition inside `if(condition)`. If True, the attached code block executes. If False, the compiler skips it and moves to the `else if` or `else` block.
- **Short-Circuiting:** In an `if -> else if -> else` chain, the compiler strictly executes **only the first** block that evaluates to True. All subsequent blocks in the chain are entirely bypassed.

Problem-Solving Deconstruction

Problem 1: Character Case Classification

1. Problem Statement & Constraints: Given a single character input, determine if it is a lowercase letter, an uppercase letter, or a numeric digit.

2. Core Intuition: Every character is mapped to an underlying integer value via the ASCII table (e.g., 'a' is 97, 'A' is 65, '0' is 48). Because these mapped values are sequential, we can use standard relational operators (`>=`, `<=`) alongside the logical AND operator (`&&`) to check if a character falls within a specific ASCII boundary.

3. Algorithmic Steps: 1. Initialize a `char` variable `ch` and capture the user input.

2. Check if `ch >= '0' AND ch <= '9'`. If True, print "numeric" and terminate block.
3. Check if `ch >= 'a' AND ch <= 'z'`. If True, print "lowercase" and terminate block.
4. Check if `ch >= 'A' AND ch <= 'Z'`. If True, print "uppercase" and terminate block.

4. Dry Run:

Input: `ch = 'k'`

- 1. Check `'k' >= '0' AND 'k' <= '9'` → `97 ≥ 48` (True) AND `107 ≤ 57` (False). Overall: False.
- 2. Check `'k' >= 'a' AND 'k' <= 'z'` → `107 ≥ 97` (True) AND `107 ≤ 122` (True). Overall: True.
- 3. Output "lowercase character".

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(1)$	Performs a maximum of 4 constant-time boolean evaluations.
Space Complexity	$O(1)$	Allocates a single byte of memory for the <code>ch</code> variable.

C++

```
#include <iostream>
using namespace std;

int main() {
    char ch;
    cout << "Enter a character value: ";
    cin >> ch;

    // Utilizing implicit ASCII value comparisons
    if (ch >= '0' && ch <= '9') {
        cout << "This is numeric character." << endl;
    }
    else if (ch >= 'a' && ch <= 'z') {
        cout << "This is lowercase character." << endl;
    }
    else {
        cout << "This is uppercase character." << endl;
    }
    return 0;
}
```

Problem 2: Sum of Even Numbers from 1 to N

- 1. **Problem Statement & Constraints:** Calculate the total sum of all strictly even integers from 1 up to a given limit N (inclusive).
- 2. **Core Intuition:** We must maintain a running accumulator (`sum`). By utilizing a `while` loop starting at an iterator of 2, we can check each number. If the number modulo 2 equals 0 (`i % 2 == 0`), we add it to our accumulator.

3. Algorithmic Steps: 1. Read input N .

2. Initialize an iterator $i = 2$ and an accumulator $sum = 0$.

3. Start while loop bounded by $i \leq n$.

4. If $i \% 2 == 0$, update accumulator: $sum = sum + i$.

5. Increment iterator: $i = i + 1$.

6. Upon loop termination, print sum .

4. Dry Run:

Input: $N = 5$

1. $i = 2$, $sum = 0$. Check $2 \leq 5$ (True). $2 \% 2 == 0$ (True). $sum = 0 + 2 = 2$. $i = 3$.

2. $i = 3$, $sum = 2$. Check $3 \leq 5$ (True). $3 \% 2 == 0$ (False). $i = 4$.

3. $i = 4$, $sum = 2$. Check $4 \leq 5$ (True). $4 \% 2 == 0$ (True). $sum = 2 + 4 = 6$. $i = 5$.

4. $i = 5$, $sum = 6$. Check $5 \leq 5$ (True). $5 \% 2 == 0$ (False). $i = 6$.

5. $i = 6$, $sum = 6$. Check $6 \leq 5$ (False). Loop ends. Output 6 .

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N)$	The loop executes linearly N times.
Space Complexity	$O(1)$	Constant variables regardless of input size.

C++

```
#include <iostream>
using namespace std;

int main() {
    int n;
    int i = 2; // Starting at first even number
    int sum = 0;

    cout << "Enter a number: ";
    cin >> n;

    while (i <= n) {
        if (i % 2 == 0) {
            sum = sum + i;
        }
        i = i + 1;
    }
    cout << "The sum of even numbers is: " << sum << endl;
```

```
    return 0;
}
```

Optimization Note: Incrementing by 2 ($i = i + 2$) removes the need for the modulo check, cutting execution time in half.

Problem 3: Fahrenheit to Celsius Converter

1. Problem Statement & Constraints: Convert a given temperature in Fahrenheit to Celsius.

2. Core Intuition: The underlying mathematical formula is:

$$= \frac{5}{9} \times (-32)$$

Crucial Trick: In C++, writing `5/9` strictly evaluates as integer division, which truncates to `0`. Multiplying anything by `0` yields `0`. We must force floating-point arithmetic by writing `5.0/9`.

3. Algorithmic Steps: 1. Initialize a `float` variable `fahrenheit` and read user input.

2. Apply the mathematical formula ensuring at least one operand is a `float` (`5.0`).

3. Store the result in a `float` `celsius` and print it.

4. Dry Run:

Input: `= 100`

1. Evaluate `(100 - 32) = 68`.

2. Evaluate `5.0 / 9 = 0.5555...`.

3. `celsius = 0.5555... * 68 = 37.7778`. Output `37.7778`.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(1)$	Performs a single static arithmetic calculation.
Space Complexity	$O(1)$	Memory footprint is strictly limited to two float variables.

C++

```
#include <iostream>
using namespace std;

int main() {
    float fahrenheit;
```

```

    cin >> fahrenheit;

    // 5.0 forces floating-point division instead of integer truncation
    float celsius = (5.0 / 9) * (fahrenheit - 32);

    cout << fahrenheit << " F = " << celsius << " C" << endl;
    return 0;
}

```

Introduction to 2D Pattern Printing

Pattern printing relies entirely on understanding nested loops. The structural framework maps visually to an X/Y coordinate grid.

- **Outer Loop (i or row):** Controls the vertical axis. Every time this loop iterates, the cursor drops to a new line (`cout << endl;`).
- **Inner Loop (j or col):** Controls the horizontal axis. Every time this loop iterates, the cursor prints characters side-by-side on the *current* row.

Pattern A: $N \times N$ Solid Star Square

Goal: Print an $N \times N$ matrix strictly filled with `*`.

Plaintext

```

***
***
***

```

- **Row Logic:** We need N rows. Outer loop runs from $i=1$ to N .
- **Column Logic:** Every row requires exactly N stars. Inner loop runs from $=1$ to N .

C++

```

#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    int i = 1;

    while (i <= n) {           // Outer Loop (Rows)
        int j = 1;
        while (j <= n) {       // Inner Loop (Columns)
            cout << "*";
            j = j + 1;
        }
    }
}

```

```

        cout << endl;        // Jump to next line after row finishes
        i = i + 1;
    }
}

```

Pattern B: Row Number Square

Goal: Print an $N \times N$ matrix where every cell displays its current row number.

Plaintext

```

111
222
333

```

- **Row Logic:** N rows needed. Outer loop runs $i=1$ to N .
- **Column Logic:** Every row requires N elements. Inner loop runs $=1$ to N .
- **Print Logic:** Instead of a `*`, we observe the number printed is identical to the row coordinate. Therefore, print `i`.

C++

```

#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    int i = 1;

    while (i <= n) {        // Outer Loop (Rows)
        int j = 1;
        while (j <= n) {    // Inner Loop (Columns)
            cout << i;      // Print current Row ID
            j = j + 1;
        }
        cout << endl;
        i = i + 1;
    }
}

```